ЛАБОРАТОРНАЯ РАБОТА 2
“ИССЛЕДОВАНИЕ ЛИНЕЙНЫХ СИСТЕМ ПРЕОБРАЗОВАНИЯ СЛУЧАЙНЫХ ПРОЦЕССОВ”

1. Цель работы

1. Получить навыки расчёта характеристик линейных систем:
- корней, полюсов и вычетов передаточной функции,
- импульсной характеристики,
- комплексного коэффициента передачи, годографа,
- АЧХ и ФЧХ системы.

2. Изучить функции прикладного пакета для расчёта указанных выше характеристик.

2. Порядок выполнения

1. Получить вариант задания, представленный в виде множеств нулей и полюсов передаточной функции, что соответствует записи вида (2).

2. Выполнить аналитическую (подготовительную, домашнюю) часть работы:
- восстановить полиномиальную форму представления функции (1);
- представить передаточную функцию в виде полюсов и вычетов (3);
- записать в явном виде весовую функцию системы (6);
- записать аналитическое представление переходной характеристики системы (7).

3. Используя специализированное программное обеспечение
- выполнить построение диаграмму нулей и полюсов Вашей системы.
- для сравнения полученных Вами коэффициентов полиномов передаточной функции с вычисленными программой коэффициентами можно использовать код;
- постройте систему полюсов и вычетов на основании коэффициентов передаточной функции, вычисленной с использованием программы, и сравните с Вашими расчётами;
- рассчитайте комплексный коэффициент передачи (4) и постройте годограф системы;

- постройте амплитудную и фазовую частотные характеристики в явном виде;

- рассчитайте импульсную характеристику h(τ), пользуясь формулой (6), и постройте её график;

- постройте график переходной характеристики системы, пользуясь формулой (7);

4. Проанализируйте полученные результаты и сделайте надлежащие содержательные выводы:

- насколько точно рассчитываются коэффициенты системы функцией zp2tf;

- насколько точно полюсы и нули системы возвращаются функцией residue(b,a);

- является ли система, вариант которой достался Вам устойчивой. Аргументируйте Ваше утверждение.

3. Вариант задачи

Таблица 1 – Вариант

№	Нули			Полюсы			
	p_1^z	p_2^z	p_3^z	p_1^p	p_2^p	p_3^p	p_4^p
42	-1,65	-	-	-2,10	-3,41	-	-

4. Ход работы

4.1. Восстановление полиномиальной формы представления функции:

$$H(p) = \frac{\big((p + 1{,}65)\big)}{(p + 3{,}41)(p + 2{,}1)} = \frac{(p + 1{,}65)}{p^2 + 5{,}51p + 7{,}161}$$

4.2. представить передаточную функцию в виде полюсов и вычетов:

$$H(p) = \frac{r_2}{\left(p - p_2^p\right)} + \frac{r_1}{\left(p - p_1^p\right)},$$

$$r_i = \frac{B(p_i^p)}{\frac{d}{dp}A'(p_i^p)}, r = 1, 2, 3, 4$$

$$r_1 = \frac{B(-2{,}1)}{\frac{d}{dp}A'(-2{,}1)} = \frac{-0{,}45}{1{,}31} = -0{,}343$$

$$r_2 = \frac{B(-3{,}41)}{\frac{d}{dp}A'(-3{,}41)} = \frac{-1{,}76}{-1{,}31} = 1{,}343$$

$$H(p) = \frac{1{,}343}{p + 3{,}41} + \frac{0{,}343}{p + 2{,}1}$$

4.3. Записать в явном виде весовую функцию системы:

$$h(\tau) = \sum_{k=1}^{n} r_k \exp\{p_k \tau\} = -0{,}46 * \exp\{-2{,}1 * \tau\} * \exp\{1{,}343 * \tau\}$$

4.4. Записать аналитическое представление переходной характеристики системы:

$$T(t) = \frac{b_0}{a_0} + \sum_{k=1}^{n} \frac{r_k}{p_k} \exp\{p_k t\}; b_0 = 1{,}65;\ a_0 = 7{,}161$$

$$T(t) = \frac{1{,}65}{7{,}161} + \frac{0{,}343}{2{,}1} * \exp\{-2{,}1 * t\} - \frac{1{,}343}{3{,}41} \exp\{-3{,}41 * \mathrm{t}\}$$

4.5. Получение системы полюсов и вычетов на основании коэффициентов передаточной функции

```
r: [-0.34351145  1.34351145]
p: [-2.1  -3.41]
k: []
```

Рисунок 1 - Вычеты и полюса, полученные программным способом с помощью библиотеки SciPy

4.6. Диаграмма нулей и полюсов системы

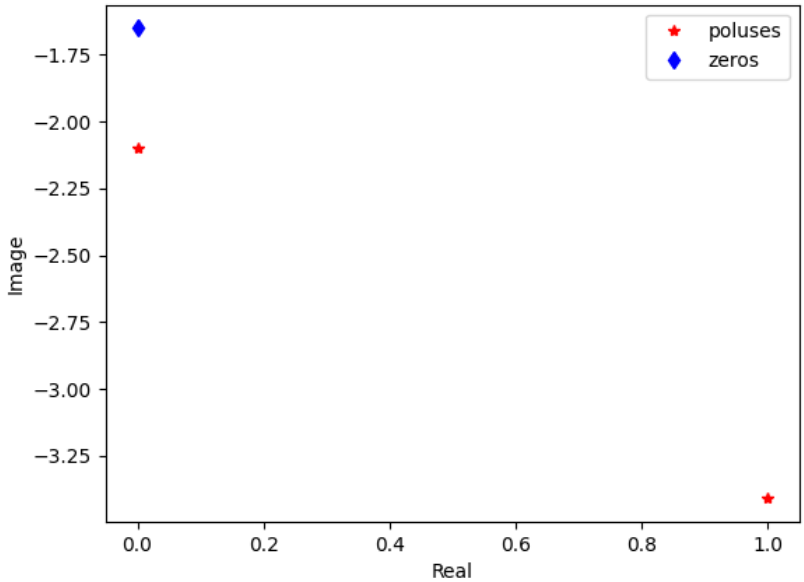

Рисунок 2 - диаграмма нулей и полюсов системы

4.7. Расчёт комплексного коэффициента передачи и построение годографа системы

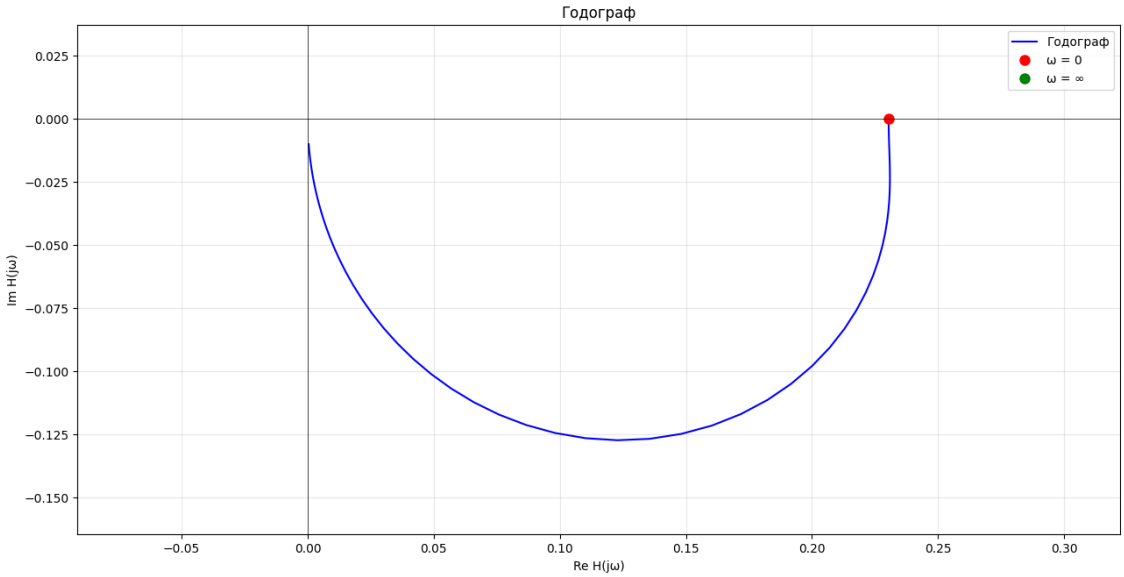

Рисунок 3 - Годограф системы

4.8. Построение графиков для амплитудной и фазовой частотных характеристик

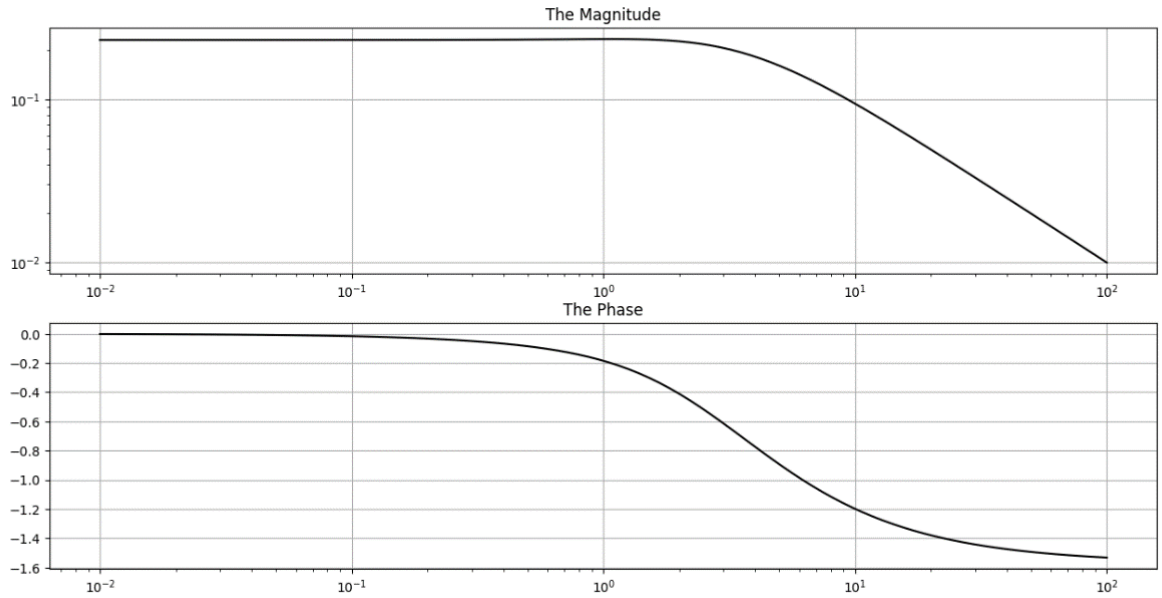

Рисунок 3 - графики для амплитудной и фазо частотных характеристик

4.9. Расчёт импульсной характеристики и построение графика.

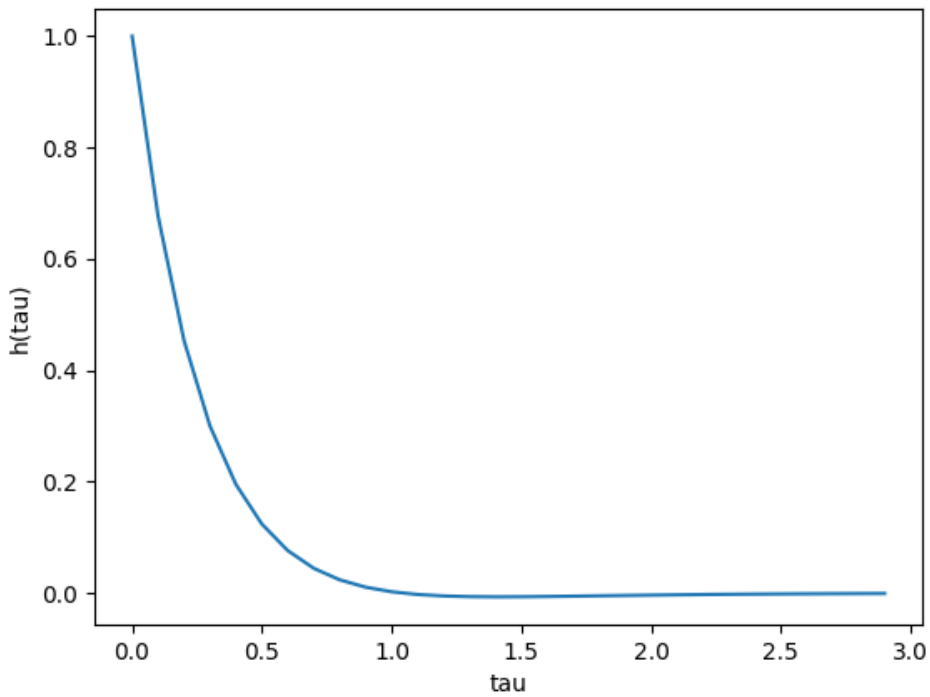

Рисунок 4 - график импульсной характеристики

4.10. Построение графика переходной характеристики системы

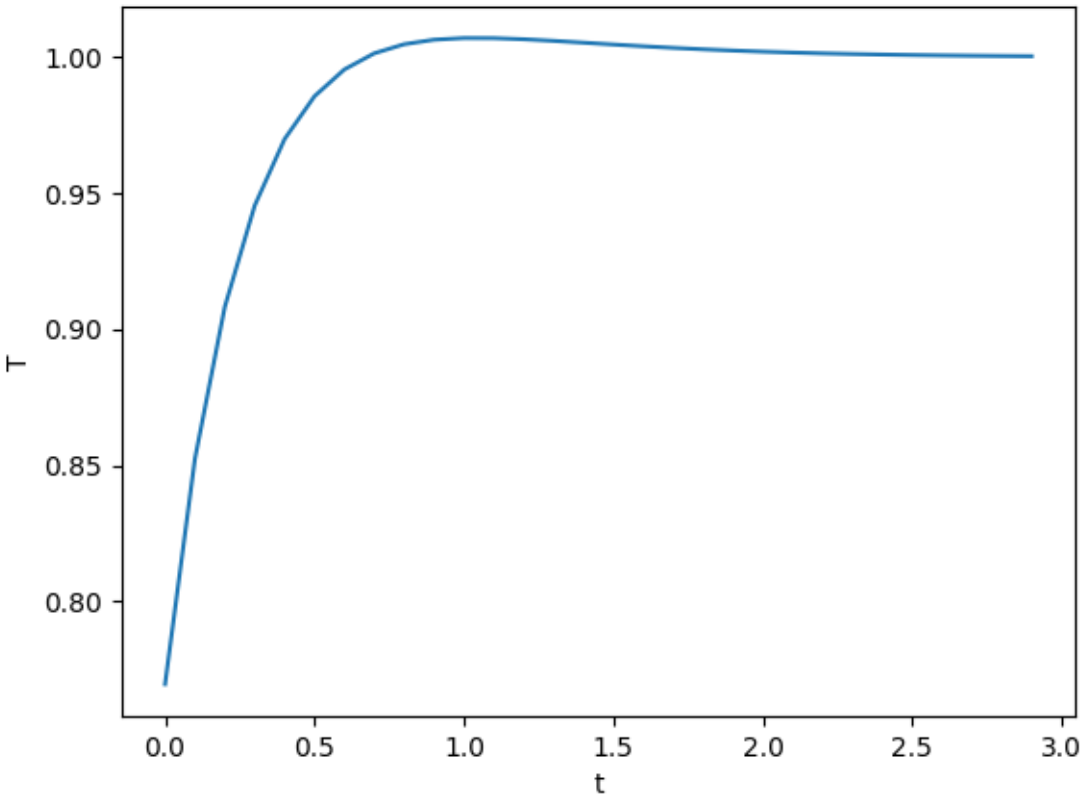

Рисунок 5 - график переходной характеристики системы

Вывод

В рамках лабораторной работы были протестированы методы определения полюсов и вычетов, вторые были найдены как программным, так и ручным методом. Тест показал, что оба метода показывают одинаковые значения вычетов, а именно -0.343 и 1.343, при этом программный метод zpk2tf был точнее, отчего цифр за плавающей точкой было больше. Был протестирован и метод residue, который возвращает полюсы и нули, было выяснено, что он возвращает те-же значения, что мы посылали в метод zpk2tf, т.е. результат на выходе функции был идентичен стартовым полюсам и нулям, что является успешным для данной функции и нашего теста. Т.к. значения действительных частей полюсов передаточной функции являются отрицательными, т.е. -2.1 и -3.41, то и система - устойчива.

ПРИЛОЖЕНИЕ А

Листинг 1 - Код для лабораторной работы

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import zpk2tf, residue, freqs

Tstart = 0
Tend = 3
Tstep = 0.1

zeros = np.array([-1.65])
poluses = np.array([-2.1, -3.41])
tau = np.array([i for i in np.arange(Tstart, Tend, Tstep)])

b, a = zpk2tf(zeros, poluses, 1)
r, p, k = residue(b, a)

def h(_tau: float):
    h = 0
    for n in range(len(r)):
        h += r[n] * np.exp(poluses[n] * _tau)
    return h

def T(time: float):
    T = 0
    for n in range(len(r)):
        T += r[n]/poluses[n] * np.exp(poluses[n] * time)
    return b[0]/a[0] + T

def makeZerosNPolusesDiagram():
    plt.plot(poluses, '*r')
    plt.plot(zeros, 'db')
    plt.xlabel("Real")
    plt.ylabel("Image")
    plt.legend(["poluses", "zeros"])
    plt.show()

def printPolynoms():
    print(f"r: {r} \np: {p}\nk: {k}")

def makeGodograph():
    w = [0, *np.logspace(-2, 2, 100), np.inf]
    _, fchx = freqs(b, a, w)
    plt.figure(figsize=(10, 8))
```

```
        plt.plot(np.real(fchx),  np.imag(fchx),  '-b',  linewidth=1.5,
label='Годограф')
     plt.plot(np.real(fchx[0]), np.imag(fchx[0]), 'ro', markersize=8,
label='ω = 0')
    plt.plot(np.real(fchx[-1]), np.imag(fchx[-1]), 'go', markersize=8,
label='ω = ∞')
    plt.axhline(y=0, color='k', linewidth=0.5)
    plt.axvline(x=0, color='k', linewidth=0.5)
    plt.grid(True, alpha=0.3)
    plt.title("Годограф")
    plt.xlabel('Re H(jω)')
    plt.ylabel('Im H(jω)')
    plt.axis('equal')
    plt.legend()
    plt.show()

    mag = np.abs(fchx)
    phase = np.angle(fchx)
    plt.subplot(2, 1, 1)
    plt.loglog(w, mag, '-k')
    plt.title("The Magnitude")
    plt.grid(True)
    plt.subplot(2, 1, 2)
    plt.semilogx(w, phase, '-k')
    plt.title("The Phase")
    plt.grid(True)
    plt.show()

def findHTau():
    array_of_h = [h(_tau) for _tau in tau]

    print("h(tau):", array_of_h)
    plt.plot(tau, array_of_h)
    plt.xlabel("tau")
    plt.ylabel("h(tau)")
    plt.show()

def findTt():
    array_of_T = [T(time) for time in tau]

    print("T(t):", array_of_T)
    plt.plot(tau, array_of_T)
    plt.xlabel("t")
    plt.ylabel("T")
    plt.show()
```

```
makeZerosNPolusesDiagram()
printPolynoms()
makeGodograph()
findHTau()
findTt()
```